

Persiapan Presentasi Docker

Mentoring: DevOps. Disusun oleh Arief Aryudi Syidik · 2 Oktober 2026 · Project: IOMS (PHP 8.2 + Apache + MySQL 8.0)

Dokumen ini memetakan 7 poin wajib dan 8 best practice level Intermediate ke bukti nyata di repo, lengkap dengan **alasan**, **risiko**, dan **bagian yang masih bisa dioptimalkan**. Status ditulis jujur sesuai isi `Dockerfile` dan `docker-compose.yml` saat ini.

1. Checklist 7 Poin Wajib

#	Poin	Cara menunjukkan		
1	Docker terpasang	<code>docker --version</code> , <code>docker compose version</code> , <code>docker run --rm hello-world</code>		
2	Tunjukkan Dockerfile	<code>cat -n Dockerfile</code> (root repo, base <code>php:8.2-apache</code>)		
3	Jelaskan instruksi utama	Bagian 2 dokumen ini		
4	build, run, Up, browser, log	Bagian 4 (Demo A)		
5	Tidak menyimpan secret di image/repo	Terpenuhi	<code>.env</code> ada di <code>.gitignore</code> dan <code>.dockerignore</code> ; hanya <code>.env.example</code> yang di-commit. Password diinjeksi saat runtime lewat <code>env_file</code> .	Default <code>root_change_me / change_me</code> di <code>compose</code> adalah placeholder, wajib diganti di luar demo. Untuk produksi, gunakan <code>Docker secrets / secret manager</code> . Nilai <code>env</code> tetap terlihat lewat <code>docker inspect</code> .
6	Compose build dari Dockerfile	<code>docker-compose.yml</code> baris 3-5: <code>build: context: . + dockerfile: Dockerfile</code> . Brief menulis <code>build: ..</code> karena <code>compose</code> -nya di subfolder; di IOMS <code>compose</code> ada di root, jadi <code>.</code> yang setara.		
7	Database sebagai service + terhubung	Service <code>mysql</code> , <code>DB_HOST: mysql</code> , bukti query dari container app (Demo B)		

2. Penjelasan Dockerfile

Baris	Instruksi	Fungsi & alasan
1	<code>FROM php:8.2-apache</code>	Base image resmi dengan PHP + Apache. Tag versi eksplisit (bukan <code>latest</code>) agar build reproducible.
5-12	<code>RUN apt-get ... docker-php-ext-install ...; a2enmod rewrite</code>	Pasang ekstensi <code>pdo_mysql / mysqli</code> dan <code>mod_rewrite</code> untuk front-controller. Satu <code>RUN</code> + <code>rm -rf /var/lib/apt/lists/*</code> agar satu layer dan cache apt tidak membengkakkan image.
15-17	<code>ENV APACHE_DOCUMENT_ROOT + sed</code>	Arahkan <code>DocumentRoot</code> ke <code>public/</code> sehingga <code>app/</code> , <code>config/</code> , <code>vendor/</code> tidak bisa diakses dari web.
20-26	Blok <code>ioms.conf</code> + <code>a2enconf</code>	<code>AllowOverride All</code> agar <code>.htaccess</code> (rewrite router) berfungsi.
29	<code>COPY --from=composer:2</code>	Ambil binary Composer dari image resmi, tanpa <code>curlphp</code> manual.
31	<code>WORKDIR /var/www/html</code>	Direktori kerja untuk instruksi berikutnya.
34-36	<code>COPY composer.json composer.lock + composer install -no-dev</code>	Urutan layer efisien : dependensi di-install sebelum source disalin, sehingga layer ini ter-cache selama file composer tidak berubah.

41	<code>COPY . .</code>	Salin source aplikasi (paling sering berubah, jadi ditaruh setelah dependensi).
44	<code>composer dump-autoload --optimize</code>	Autoloader teroptimasi terhadap source final.
46-48	<code>mkdir/chown/chmod public/uploads</code>	Folder upload bisa ditulis <code>www-data</code> .
50	<code>EXPOSE 80</code>	Dokumentasi port; publish tetap lewat <code>-p/Compose</code> .

3. Audit 8 Best Practice Intermediate

#	Best practice	Status	Bukti & alasan	Risiko / perbaikan
1	Base image dengan tag versi jelas	Terpenuhi	<code>php:8.2-apache</code> , <code>mysql:8.0</code> , <code>composer:2</code> . Tidak ada <code>latest</code> .	Tag minor masih "bergerak" (patch baru masuk saat rebuild). Opsi: pin patch (<code>php:8.2.x-apache</code>) atau digest untuk reproduksi penuh.
2	Urutan layer efisien	Terpenuhi	Baris 34-36 (<code>composer files + install</code>) sebelum baris 41 (<code>COPY . .</code>); <code>apt</code> dalam satu <code>RUN</code> .	Perubahan source tidak memicu install ulang dependensi. Peningkatan: multi-stage agar <code>git / unzip/composer</code> tidak ikut ke image akhir.
3	Menggunakan <code>.dockerignore</code>	Terpenuhi	File <code>.dockerignore</code> di root: mengecualikan <code>.git</code> , <code>.env*</code> (kecuali <code>.env.example</code>), <code>vendor</code> , <code>tests</code> , <code>docs</code> , <code>log</code> , dan isi <code>public/uploads</code> .	Build context lebih kecil, cache lebih stabil, dan <code>.env</code> tidak masuk layer image (terverifikasi: <code>ls -a</code> di image tidak menampilkan <code>.env / .git</code>). Konsekuensi: <code>vendor/</code> <code>host</code> tidak ikut, sehingga <code>composer install</code> di build harus berhasil (butuh jaringan).
4	Konfigurasi via environment variable	Terpenuhi	Compose memakai <code>env_file: .env</code> dan <code>environment (DB_HOST, DB_PORT)</code> ; MySQL memakai <code>\${DB_DATABASE:-ioms}</code> <code>dst</code> .	Aplikasi membaca config dari env sehingga image yang sama bisa dipakai di dev/staging/prod tanpa rebuild.
5	Tidak menyimpan secret di image/repo	Sebagian	<code>.env</code> ada di <code>.gitignore</code> ; hanya <code>.env.example</code> yang di-commit. Password diinjeksi saat runtime.	(a) Tanpa <code>.dockerignore</code> , <code>.env</code> bisa terbawa ke image (lihat #3). (b) Default <code>root_change_me / change_me</code> di compose adalah placeholder, bukan secret asli, tapi wajib diganti di luar demo. (c) Untuk produksi, gunakan Docker secrets / secret manager.
6	App & database via Docker Compose	Terpenuhi	Dua service: <code>app</code> dan <code>mysql</code> dalam satu network default; app memakai hostname <code>mysql</code> .	Satu perintah <code>docker compose up</code> menghidupkan seluruh stack; urutan start dijamin <code>depends_on: service_healthy</code> .
7	Volume untuk data yang perlu dipertahankan	Terpenuhi	<code>db_data:/var/lib/mysql</code> dan <code>uploads_data:/var/www/html/public/uploads</code> (named volume).	Data tetap ada setelah <code>docker compose down</code> ; hilang hanya jika <code>down -v</code> . Bind mount <code>./:/var/www/html</code> hanya cocok untuk development (lihat bagian 5).
8	Healthcheck pada service relevan	Terpenuhi	MySQL: <code>mysqladmin ping</code> . App: <code>curl -fsS http://localhost/</code> (interval 10s, retries 5, start_period 15s). <code>curl</code> sudah ada di base image. Terverifikasi: <code>docker compose ps</code> menampilkan <code>Up (healthy)</code> untuk keduanya.	Status "Up" kini benar-benar menandakan Apache+PHP merespons. Healthcheck MySQL membawa password di perintah (terlihat di <code>docker inspect</code>); alternatif: baca dari env di dalam container. Healthcheck app hanya cek HTTP, belum cek koneksi DB.

Ringkas: 8 dari 8 terpenuhi. Poin yang masih bisa dioptimalkan (bukan kewajiban): pin tag patch/digest, multi-stage build, user non-root, dan pemisahan konfigurasi dev/prod (bagian 5).

4. Demo yang Dijalankan

Demo A: Dockerfile (poin 4)

```

docker build -t ioms-app:demo .
docker network create ioms-net
docker run -d --name ioms-db --network ioms-net \
-e MYSQL_ROOT_PASSWORD=root_change_me -e MYSQL_DATABASE=ioms \
-e MYSQL_USER=ioms_user -e MYSQL_PASSWORD=change_me \
-v "$PWD/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro" \
-v "$PWD/database/seed.sql:/docker-entrypoint-initdb.d/02-seed.sql:ro" mysql:8.0
# tunggu 20-30 detik sampai schema selesai di-load
docker run -d --name ioms-demo --network ioms-net -p 8081:80 \
--env-file .env -e DB_HOST=ioms-db -e DB_PORT=3306 ioms-app:demo
docker ps --filter name=ioms          # status Up
open http://localhost:8081            # browser
docker logs ioms-demo 2>&1 | grep -i fatal || echo "tidak ada fatal error"
docker rm -f ioms-demo ioms-db; docker network rm ioms-net # bersihkan

```

Demo B: Docker Compose (poin 5-7)

```

docker compose up --build -d
docker compose ps          # app Up, mysql Up (healthy)
open http://localhost:8080
docker compose logs app | grep -i fatal || echo "tidak ada fatal error"
docker compose exec mysql mysql -uioms_user -pchange_me ioms -e "SHOW TABLES; SELECT COUNT(*) FROM users;"
docker compose exec app php -r 'require "vendor/autoload.php"; App\Core\Env::load(".env"); echo App\Core\Database::connection()->query("SELECT DATABASE()")->fetchColumn(), PHP_EOL;'
# bukti akhir: login di browser (user dibaca dari MySQL)

```

Bukti best practice (tunjukkan saat ditanya)

```

docker compose config | sed -n 1,40p          # konfigurasi hasil resolve env
docker volume ls | grep ioms                 # named volume db_data, uploads_data
docker compose down && docker compose up -d   # data tetap ada (volume)
docker inspect --format '{{.State.Health.Status}}' $(docker compose ps -q app mysql)
docker history ioms-app:local                 # tampilkan layer & ukuran
docker run --rm ioms-app:local ls -la /var/www/html | grep -E "^\.(env|git)$" || echo ".env tidak ada di image"

```

Verifikasi: perintah terakhir harus menampilkan ".env tidak ada di image" karena `.dockerignore` mengecualikan `.env`. Jika tidak, build ulang tanpa cache: `docker compose build --no-cache`.

5. Alasan Konfigurasi, Risiko, dan Optimasi

Keputusan	Alasan	Risiko	Optimasi
Image php:8.2-apache single-stage	Sederhana, ekstensi mudah dipasang, cocok untuk app PHP monolit.	Image berisi git, unzip, Composer: ukuran lebih besar dan permukaan serangan lebih luas.	Multi-stage: stage composer untuk install dependensi, stage akhir hanya runtime. Hapus git / unzip dari final.
composer install ... echo "..."	Build tetap jalan di lingkungan offline dengan vendor/ dari host.	Kegagalan install tersembunyi; build "hijau" tetapi image bisa rusak.	Hapus fallback, biarkan build gagal bila dependensi gagal; andalkan vendor/ dari image, bukan host.
Bind mount ./var/www/html	Edit kode langsung terlihat tanpa rebuild (development).	Menimpa isi image, sehingga yang jalan bukan artefak yang di-build; butuh vendor/ di host; permission host/container bisa bentrok.	Pisahkan docker-compose.override.yml untuk dev; compose utama tanpa bind mount.
Port MySQL 3307:3306 di-publish	Memudahkan akses dari client SQL host.	DB terekspos di host/jaringan.	Hapus publish di produksi (app cukup lewat network internal) atau bind ke 127.0.0.1:3307.
Container berjalan sebagai root (tanpa USER)	Apache butuh bind port 80 saat start, lalu menurunkan worker ke www-data.	Proses master tetap root.	Gunakan image/port non-privileged (mis. listen 8080) dengan USER www-data.
restart: unless-stopped	Service naik lagi otomatis setelah crash/reboot.	Crash loop bisa menutupi bug.	Pantau lewat healthcheck dan log.

6. Penjelasan Hubungan App & Database

Compose membuat network default; setiap service dapat dijangkau lewat **nama service**. Karena itu `DB_HOST: mysql` (bukan `localhost`, yang berarti container app itu sendiri). Aplikasi membaca `DB_*` dari environment di `app/Core/Database.php` lalu membuka koneksi PDO ke `mysql:3306` (port internal), sedangkan `3307` hanya untuk akses dari host. `depends_on` dengan `condition: service_healthy` memastikan app baru start setelah MySQL lolos healthcheck, dan skrip `schema.sql` + `seed.sql` di `docker-entrypoint-initdb.d` hanya dijalankan saat volume `db_data` masih kosong.

7. Langkah Menjalankan Aplikasi (ringkasan)

```
cp .env.example .env      # isi DB_* sesuai kebutuhan
docker compose up --build -d
docker compose ps
open http://localhost:8080
docker compose down      # tambahkan -v hanya jika ingin menghapus data
```

8. Urutan Presentasi (±15 menit)

Menit	Langkah
1	Versi Docker, hello-world (poin 1)
2-5	Dockerfile dan penjelasan instruksi, termasuk urutan layer (poin 2-3, BP 1-2)
6-9	Demo A: build, run, <code>docker ps</code> , browser, log (poin 4)
10-13	Demo B: Compose, service mysql, volume, healthcheck, <code>env_file</code> , koneksi DB (poin 5-7, BP 3-8)
14-15	Bagian yang masih bisa dioptimalkan (multi-stage, non-root, dev/prod split)

9. Antisipasi Pertanyaan

Pertanyaan	Jawaban singkat
Kenapa <code>COPY</code> composer.* dulu?	Layer cache: dependensi jarang berubah, source sering; install tidak diulang tiap edit kode.
Kenapa tidak <code>latest</code> ?	Tidak reproducible; update diam-diam dapat merusak build.
Apa beda <code>EXPOSE</code> dan <code>ports</code> ?	<code>EXPOSE</code> hanya dokumentasi; <code>ports</code> / <code>-p</code> yang benar-benar mem-publish ke host.
Kenapa pakai named volume?	Dikelola Docker, tidak bergantung path host, data bertahan setelah container dihapus.
Bagaimana mengelola secret di produksi?	Jangan di image/repo; pakai env runtime dari secret manager atau Docker/Swarm secrets.
Apa yang akan Anda perbaiki lebih dulu?	<code>.dockerignore</code> dan healthcheck app sudah ditambahkan. Berikutnya: multi-stage build (buang git/unzip/composer dari image akhir), user non-root, dan hapus bind mount serta publish port MySQL di produksi.